Deriving Sound Test Scripts from Requirements Written in a Controlled Natural Language

Filipe Arruda[1(✉)], Flávia Barros[2], and Augusto Sampaio[2]

[1] Centro de Tecnologias Estratégicas do Nordeste, Recife, Brazil
filipe.arruda@cetene.gov.br
[2] Centro de Informática, Universidade Federal de Pernambuco, Recife, Brazil
{fab,acas}@cin.ufpe.br

Abstract. In an industrial context, ad-hoc/manual testing strategies, using natural language, still seem to be highly prevalent since natural language descriptions are, more likely, easier to understand. Still, the lack of rigor can generate inaccurate tests. Aligned with other modern approaches, we promote the use of natural language descriptions with rigorously defined underlying semantics. As a distinguished feature of our approach, we cover the entire (direct engineering) testing process, from requirements to manual or automated test cases generated automatically. Requirements written in a controlled natural language are parsed, and their semantics are automatically modeled using the CSP process algebra. To address soundness and deal with different abstraction levels, we formalize the concept of a domain model, in which additional information, such as hierarchical composition and a dependence relation among test steps, is defined. Then, sound test cases are generated from the inferred scenarios using the *cspio* conformance relation. These test cases, still expressed in CSP, can then be linearized back to natural language to allow manual execution or directly translated into test scripts for automated execution.

Keywords: Formal Semantics · Controlled Natural Language · Test Case Consistency · Domain Model

1 Introduction

Typically, test cases (TCs) are automated as scripts within a testing framework such as JUnit[1], Pytest[2], or Selenium[3]. They are important resources to verify whether an implementation conforms to the system requirements. The adoption of test automation tools or techniques reduces the cost of test cycles [15]. However, despite this global reduction, the initial implementation cost tends to be

[1] https://junit.org/.
[2] https://pytest.org/.
[3] https://www.selenium.dev/.

M. H. ter Beek and L. Teixeira (Eds.): SBMF 2025, LNCS 16363, pp. 101–118, 2026.
https://doi.org/10.1007/978-3-032-12086-1_6

higher with the use of automation practices, and maintenance can be problematic. Therefore, test scripts should be designed in a way that encourages reuse, becoming easier to maintain and to remain functional despite constant changes on the System Under Test (SUT). In this way, automation becomes an efficient and sustainable solution [10].

The traditional testing process faces several issues: abstraction gap and traceability challenges when linking artifacts like requirements, test cases, and scripts across different abstraction levels and roles; heterogeneous notations that require varied expertise and hinder consistent maintenance; inconsistencies between abstract requirements and concrete tests due to incomplete setups, unmet dependencies, or missing input mappings; inefficiencies in execution time and order when scripts are made self-contained instead of optimizing execution sequences; internationalization problems that introduce ambiguity and misinterpretation, especially with translations; technology lock-in risks from tying the process to specific, fast-changing tools; legibility concerns for novice testers unfamiliar with complex notations; and duplication of artifacts caused by poor traceability.

The aforementioned issues can all be traced back to the lack of a mechanized analysis using precise notations and formal strategies to test case generation and automation. The seminal work of Gaudel [9] argues that testing can be formal, too. From this perspective, there are several test generation strategies and theories based on formal conformance relations, such as *ioco* [19] and *conf* [4]. However, even with a rigorous formal approach, some maintenance issues still arise, such as execution inconsistencies and over-detailed models that need to be updated in every development iteration. Considering this context, the leading question that guides our research is *how to generate sound, consistent and executable test scripts based on requirements written in natural language?* To answer this question, we consider some related challenges: 1) How to process natural language descriptions; 2) How to verify the soundness of derived test cases; and 3) What practical strategy can be used to generate tests that can be interpreted directly by a test driver without demanding over-detailed requirements. The proposed solutions for these challenges are listed next.

- A unified Controlled Natural Language (CNL) to describe requirements, domain model and test cases with a well-defined syntax and precise semantics;
- A denotational semantics in Communicating sequential processes (CSP) for requirements and domain models written in compliance with the CNL.
- A formal support for further specification, *via* domain model, aside from the original requirements, while preserving the underlying behavior.
- Soundness verification by modelling the Implementation Under Test (IUT) as a CSP process, using *CSP Input-Output Conformance (cspio)* [14] as conformance relation.
- A formal definition for test consistency, a property that ensures that test cases, formerly abstract, can be directly executed in an implementation without inconclusive or inconsistent results.
- Tools implementing the direct engineering from requirements, including the automation of the generated test cases.

In the following section, we propose a CNL and present its grammar, parsing rules, and the framework chosen for implementation. Section 3 introduces CSP and the semantic rules for translating a syntactic tree, obtained after parsing requirements and domain model that comply with the CNL, into a CSP model. In Sect. 4, we introduce the *ioco* testing theory, together with its CSP adaptation (cspio). Then, we discuss how to generate sound and consistent test cases by means of CSP refinement assertions. With the resulting traces, we show how to translate them back to natural language and how to map them into automated scripts. Finally, Sect. 5 discusses related work, while Sect. 6 presents our conclusions, summarizes our contributions, and discusses next steps. In particular, future research could explore two integration strategies, both based on Large Language Models (LLMs). One possibility is a mapping approach to infer the frames and slots directly from free-style natural language. An alternative could be translating free-style requirements into grammatically and semantically valid CNL constructs. This would preserve the interpretability and verifiability of the CNL framework while automating its most labor-intensive aspects.

2 Parsing CNL-Compliant Requirements

The pursuit of an integrated testing process eventually has to face the following question: *which notation should be adopted?* As it is further discussed in Sect. 5, there are several proposed frameworks which adopt *ad-hoc* notations to represent the requirements and the generated artifacts. However, one major downfall of adopting such notations is finding (or training) specialists. Thus, the answer to our initial question may indicate the use of a widely known notation: our written natural language since it is ubiquitous and expressive. While natural language is a widely known notation, because of its ambiguous nature, it may not be deterministically interpreted by an algorithm. Then, there is no straightforward way to define a meaningful mapping between textual descriptions and their semantic representations. This lack of precision typically leads to, for instance, an abstract gap between sentences and the actual corresponding execution code. In this light, we propose the use of an CNL, allowing the users to write artifacts in English while certifying that a standard is being followed, leading to a deterministic machine interpretation.

2.1 Frame Structure

We adopted a framework for knowledge representation that allows computer interpretation. The chosen schema is heavily built upon the concept of a *frame*, which is a structure to store data about a stereotyped situation, as defined in [13]. Our work was also inspired by the linguistic approach to semantic representation known as Case Frames [8], which focus on the verb. Each verb defines its own frame, as verb complements may vary.

These frames contain prefixed *slots* that, when filled, represent an instance of a specific situation. Each slot holds a different purpose. Consequently, there

Table 1. Frame example

Required (Static)			Extra (Dynamic)		
Agent	Operation	Patient	Sender	Receiver	Title
(User)	Send	Email Message	filipe.arruda@cetene.gov.br	acas@cin.ufpe.br	Smartest

can be a specific rule (or set of rules) for filling each slot. Because our purpose is to ultimately represent test actions, our frames should convey elements that resemble test steps. In our context, the "agent" slot defaults to the user, since our frames aim to describe how a user (tester) can interact with the SUT. The verb slot is named as 'operation', and its immediate complement is always the 'patient' (see Table 1). We also adapted the Frame theory by dividing the slots into two categories: Required and Extra. The former must be present in every frame, acting as a unique identifier. The latter, instead, does not define the action intention, but all other dynamic properties and modifiers of a situation. This separation allows us to use the required slots as unique identifiers (when mapping to script methods) and pass the dynamic slot values as arguments.

2.2 Syntax

Our main goal when designing the CNL is to write any test artifact using the same subset of natural language, following the same rules. To provide a general idea of which sentences comply with our CNL, we present in [3] an overview of the grammar (excerpt) using the Extended Backus-Naur Form (EBNF) notation.

To simplify the presentation of the syntax, we begin by illustrating a simple sentence that complies with intermediate production rules. For instance, the production rule for Actions states that it is required to have an Operation and a Patient. Thus, the following basic sentence would be successfully recognized: *A message is sent.* The word "message" would be the Patient while "send" would be the Operation. Additionally, an action can have Agent, Modality, Polarity and a Predicative Qualifier. For illustration, a sentence containing all these slots is: *The user must not send a message early.* In this case, we incremented the initial sentence by adding the "user" as Agent, a Required Modality (represented by the modal verb "must"), the Negative Polarity and a Predicative Qualifier (adjective "early"). Also, the patient slot can be further incremented. For instance, we could add an anaphoric reference by replacing "message" for the pronoun "it". Also, we could add an attributive adjective for the object in question, or even a quantifier. Finally, we could use coordinating conjunctions to join two patients and form a more complex one (noun phrase coordination).

To describe requirements, sentences may have a Circumstance, which can be a Single (or recurrent) Event or a Condition. Each Circumstance has a corresponding preceding conjunction, namely "when", "if" and "until" (and their synonyms). For instance, the following sentence has a circumstance: *When the button is pressed then a message is sent.* Ultimately, both Circumstances and

Statements are described by means of Actions, allowing us to use a single structure for conditions and tasks, More complex configurations can be found by exploring the other production rules, but the above examples represent the core language we defined. Implementation details are discussed next.

2.3 CNL Implementation

The CNL requirements and Backus-Naur Form (BNF) both give an idea of what sentences comply with our standard. However, other technical and functional issues arise when implementing these constraints.

To implement the parser, we used The Grammatical Framework (GF). It is both a theoretical framework and a special-purpose programming language for the description of natural languages [2]. It was designed to have an out-of-the-box support for the complexities found in natural languages [16]. Every GF grammar is composed of one abstract syntax and one or more concrete syntaxes. Abstract syntaxes model a specific application domain (similarly to ontologies), whereas concrete syntaxes are language-dependent and thus the latter encode a particular idiom. Because there is a separation between the syntaxes, the abstract one can be used as interlingua between multiple concrete languages [2].

The development of domain-specific application grammars, such as the one defined in this work, usually reuses a subset of the natural language syntax and lexicon to reduce ambiguity. To ease this burden, GF provides a Resource Grammar Library (RGL), which covers comprehensive morphologies and syntactic structures from more than 20 languages [11]. In this light, instead of defining from scratch the inner workings of a natural language, we can reuse the mature syntactic and morphologic paradigms, detailed by specialists, already present in the RGL. For instance, to create a full-fledged sentence using RGL, we could use the mkS function with the following signature: $mkS\ (Tense) \rightarrow (Ant) \rightarrow (Pol) \rightarrow Cl \rightarrow S$. This function must receive a tense (conditional, future, past or present), an anteriority (anterior or simultaneous), polarity (positive or negative), and a declarative clause to be adapted.

In our scenario, not only do we need to parse a sentence but also to manipulate the parse tree in order to linearize it in different ways. For instance, a test step must always be in the imperative mood, while requirements may be presented in the indicative mood. Even though the action (identified by its frame slots) remains the same, its linearization may differ depending on the artifact.

3 Formal Semantics for CNL-Compliant Specifications

We define a formal semantics for compliant requirement models via mapping into CSP models. More specifically, we map actions and circumstances into CSP events and processes. We then use the FDR model checking tool to automate conformance verification and Test Case (TC) generation, via process refinement checking. A distinguishing feature of using a process algebra like CSP for this

purpose is abstraction. Since test generation is expressed in terms of counterexamples obtained from refinement checking using FDR, rather than by defining ad-hoc algorithms for each modeling formalism [14], we reason at a purely process algebraic level, agnostic to the structure of a particular model. As a result, there is no need for manipulation of state spaces or control flow, which are typical in algorithmic test generation approaches based on operational models such as Labelled Transition System (LTS) or Finite-State Machine (FSM). Also, in a process algebraic context, it is possible to extend an approach that consider ony control flow with data and timed aspects, for instance.

Communicating Sequential Processes (CSP) is a formal notation for modeling concurrent systems using algebraic, denotational, and operational approaches. Its core abstractions are processes and events. An event is a basic modeling unit representing any given situation. The abstraction level depends on which behaviors are relevant to the model. The other basic elements of CSP are the fundamental processes *Stop* and *Skip*, which model the absence of communication (deadlock) and successful termination, respectively. In addition to those core elements, one can use a rich repertoire of algebraic operators [17].

3.1 Semantics of Requirements

Because we aim to define a formal semantics for the proposed CNL requirements in CSP, in order to be able to reason about soundness and consistency we must adopt an appropriate mapping that reflects the dynamics of interacting with a SUT to observe its responses. In this light, we represent the requirements in terms of input and output events. The testing theory is discussed in Sect. 4.

Test actions are the building blocks of test steps, domain models and requirements. Then, since all artifacts are just different kinds of compositions of these actions, we provide a mapping between the test action elements and CSP processes. Due to the testing theory we adopt, there must be a distinction between input and output events in a specification. The intuition for interpreting a requirement is simple: every action from a requirement circumstance is mapped to an input event, while the actions within a statement are translated into outputs.

Here we present the initial semantic rule that translates the specifications into CSP processes and events. All semantic rules are detailed in the extended version [3]. It is worth mentioning that, for simplicity, we assume that all necessary channels and auxiliary sets are declared. The initial and more abstract artifact is the feature document, in which the requirements are listed. Each requirement, in turn, can be represented by one or more normalized sentences. The metalanguage expressions used to define the semantic rules are underlined and highlighted with a different color (gray). We also use meta keywords to define auxiliary and local functions (*let...within*) and other variables (*where*). The symbol $\widehat{=}$ denotes a function definition.

Rule 1. Feature ⟦ feature: Feature ⟧ =

let list(current, remaining) ≙
  if #remaining = 0
    ⟦ current ⟧
  else
    ⟦ current ⟧ [+ $\alpha_{current} \cap \alpha_{head\ remaining}$ +] list(head remaining, tail remaining)
within
  REQ = list(head sentences, tail sentences)
where
  sentences = normalize(feature.sentences)

Rule 1 shows that a feature is composed of a sequence of sentences. Each sentence is then semantically interpreted in a separate rule. Finally, the final specification, called *REQ*, is defined by using the synchronizing external choice operator between all individual sentences. Each sentence usually represents a single requirement, but there are cases where it can represent more than one. This is why we have to normalize them by searching the sentence for disjunctions.

Each normalized sentence (by splitting disjunctions when necessary) is mapped to a CSP process by converting its circumstances (inputs) and statements (outputs) into event sequences. This conversion also assigns a unique identifier to each action by concatenating its slot values, and prefixes it with *i_* or *o_* according to whether it represents an input or an output event. The result is a precise event trace for each sentence, which the FEATURE rule then composes using the synchronizing external choice operator to form the complete specification. To illustrate these semantic rules presented in the previous section, consider the feature: *i) A message must be sent after the option is enabled. ii) When the option is enabled the main screen should be shown.*
By applying Rule 1, assuming that $sentences = [sentence_i, sentence_{ii}]$, *REQ* is defined as the resulting process of applying the synchronizing external choice operator over both processes, as shown below:

Example 1. REQ = i_enableoption → o_sendmessage → *Skip*
  [+ {*i_enableoption*} +]
  i_enableoption → o_showmainscreen → *Skip*

3.2 Semantics of Domain Models

While requirements describe what should be implemented and tested, domain models give details on how. It is especially relevant for test engineers to give concrete implementation details when creating test cases so manual testers and the automation team can execute the tests without guessing undocumented behaviors. These concrete details can be expressed through associations between actions which together form a domain model.

The following description presents a sample domain model related to "Sending a message". There is a well-known *dependency* between "Send a message" and "Activate a connection", requiring the connection to be activated before sending a message. Conversely, "Activating the Airplane Mode" *cancels* any action of "Activating a connection". Thus, if airplane mode is activated after the connection, the connection must be activated again before attempting to send a message. The *instantiation* relation represents concrete and alternative ways to execute an action; for instance, to "Activate a connection", one can "Activate the WiFi" or "Activate the 4G". Finally, "Open the app", "Write a message", and "Submit the message" describe the ordered steps to "Send a message", and these actions can also be further detailed.

Rule 2 gives an overview of the main events and processes used to model the domain. We define the *DOMAIN* process that uses a replicated external choice operator over all main inputs, followed by a recursive call. *domain_main_inputs* includes only the main inputs, i.e., the first (source) input event of a relation. For instance, by envisioning the domain model as a graph, a dependency between two actions could be represented by the edge *i_sendmessage* → *i_turnwifion* ("send message" depends on "turning the wifi on"), in which the first node would be the main input.

Rule 2. Domain ⟦ domain: Domain ⟧ =

```
DOMAIN = [] main_input: domain_main_inputs @ EXECUTE(main_input); DOMAIN
EXECUTE(x) = (INJECT(x) [] NOTINJECT(x)); CANCELS(x)
INJECT(x) = DEP(x); COMPOSITE(x); INJECTED(x)
COMPOSITE(x) = execute.x -> INSTANTIATE(x); x -> executed.x -> SKIP
⟦ domain.dependencies ⟧
⟦ domain.cancellations ⟧
⟦ domain.details ⟧
⟦ domain.instantiations ⟧
⟦ domain.consistencyChoices ⟧
```

Then, we have the definition of *EXECUTE* which, in words, describes the steps of how to consistently execute an action by considering its details. Its definition begins by presenting a choice between two distinct parameterized processes *INJECT* and *NOTINJECT*. It introduces the possibility of modeling what happens when the execution is consistent and when it is not. The latter is especially important when testing for setup error scenarios. By diving on the *INJECT* definition, we identify a sequential composition of *DEP*, *COMPOSITE* and INJECTED. The processes *DEP* and INJECTED are placeholders for describing what should be executed before and after any given action. *DEP* is only defined for actions that have dependencies. In its definition, discussed in Rule 3, the actions that should be executed before x will be declared. *COMPOSITE* is

a process for detailing how to execute the input x. Its definition shows the auxiliary events *execute* and *executed*, to mark exactly before and after the action execution. Other processes can add events related to the execution of x by synchronizing with these auxiliary marks. If x is an abstract input, for instance, the concrete actions will be recursively listed. It is worth mentioning that, between these auxiliary events, instead of just synchronizing with x we call the process *INSTANTIATE* which is a placeholder for the cases when an abstract action can be executed in different but equivalent ways.

After the definition of these core processes, we have the characterization of all actual dependencies, cancellations, details, instantiations and consistency choices. Rule 3 shows that, for all dependencies present on the domain model, we define an "instance" of *DEP*, pattern matching with the given event *main*. The definition of *DEP* shows that there are two alternatives: the setup is already executed or it should be executed. We rely on the FDR analysis mechanism to give the shortest trace, thus not including re-execution of dependencies when it is not necessary. The *VERIFY_DEP* makes clear that if the dependency was already executed (via *EXECUTE*) then it will synchronize with the auxiliary event *isexecuted*. Finally, *DEP*(_) is defined for all the other cases when an action does not have dependencies.

Rule 3. Dependencies ⟦ dependencies: DependencyList ⟧ =

```
for main, deps @ dependencies
  DEP(main.id) = for dep in deps
    (VERIFY_DEP(dep.id) [] EXECUTE(dep.id));
DEP(_) = SKIP
VERIFY_DEP(x) = isexecuted.x -> SKIP
```

While all rules are detailed in an extended version [3], we give a brief description of the remaining rules next. *Cancellations* describe which events must be re-executed when a given main action is performed, covering incompatible actions (e.g., logging out cancels a previous login). *Details* specify the concrete steps to execute an abstract action as a sequence of sub-actions, inspired by the composite design pattern. *Instantiations* are similar, but define an action in terms of equivalent alternatives, allowing flexibility in execution (e.g., activating WiFi or 4G to establish a connection). Consistency choices model both success scenarios and what should happen when setup actions are missing, enabling the specification of explicit behaviors for inconsistent executions and reducing inconclusive analysis results.

To illustrate the semantic rules for domain models presented above, we get the partial CSP model as follows:

$$DEP(i\_sendmessage) = (VERIFY\_DEP(i\_activatedata) \ [] \ EXECUTE(i\_activatedata))$$
$$DEP(\_) = Skip$$
$$CANCELS(i\_activateairplanemode) = cancels.i\_activateconnection \rightarrow Skip$$
$$CANCELS(\_) = Skip$$
$$DETAILS(i\_sendmessage) = EXECUTE(i\_openapp);\ EXECUTE(i\_writemessage);\ EXECUTE(i\_submitmessage)$$
$$DETAILS(\_) = Skip$$
$$INSTANTIATE(i\_activateconnection) = EXECUTE(i\_activatewifi) \ [] \ EXECUTE(i\_activatefourg)$$
$$INSTANTIATE(x) = DETAILS(x)$$

Since the domain model and requirements are defined in terms of CSP processes, we can now leverage generation mechanisms built upon refinement checkers to generate sound and consistent tests. A fitting generation mechanism is discussed in the next section.

4 Sound and Consistent TC Generation and Automation

Because test completeness is not feasible, we can not assume that a System Under Test (SUT) is correct only because some test cases passed. However, if we guarantee that whenever a test fails, then the SUT is not conformant to the requirements, we say that this particular test is sound. The approach to generate sound test cases is to define a formal conformance relation between implementation and specification, from which we derive valid test cases.

4.1 Test Generation

Based on a conformance relation, test cases can be automatically generated from the input test model using algorithms which ensure that the generated tests satisfy relevant properties, such as *soundness*. Informally, soundness means that if an IUT fails a test case, then the IUT does not conform to the model from which the test was generated. The *ioco* (Input-Output Conformance) [19] is one of the most widely used conformance relations. It is based on Input-Output LTS (LTS) that in turn are a special class of Labelled Transition System (LTS). IOLTS events, unlike LTS, are partitioned into input and output events.

Definition 1 (ioco).
$i\ ioco\ s \mathrel{\widehat{=}} \forall \sigma \in straces(s) \bullet out(\Delta_i, \sigma) \subseteq out(\Delta_s, \sigma)$
$where\ out(X, \sigma) = \{e : O_\delta \mid \sigma \frown \langle e \rangle \in straces(X)\}$

Definition 1 states that an implementation i conforms to a given specification Δ_s when the set of observed outputs after "performing" any suspension

trace (σ) in i is a subset of the observed outputs after the same trace (σ) in Δ_s. Suspension traces, in addition to the original concept of traces, include the observation of quiescence [6]. The symbol Δ_x, where x is any LTS, represents x behavior after adding δ in every state that manifests quiescence. Likewise, the symbol δ annotated on the set O indicates that it includes quiescence. Quiescence represents the lack of observable behavior, as in deadlocks, livelocks, and output locks. Considering this definition, we can establish that *iolts2* ioco *iolts1* but the opposite is not true, since *o_dialog* is not an output observable in *iolts2* after the trace $\langle i\_send \rangle$: $out(iolts1, \langle i\_send \rangle) \nsubseteq out(iolts2, \langle i\_send \rangle)$

To allow an automated conformance verification and further test case generation via model checking, without relying on ad-hoc algorithms, we will use a process algebraic characterization of the ioco relation in CSP called *CSP Input-Output Conformance (cspio)*. This new relation also assumes that the events are partitioned into inputs and outputs and that the Implementation Under Test (IUT) can be modelled as a CSP process. Similar to *ioco*, any given IUT conforms to a specification S if, after performing the same available traces, the outputs from IUT are a subset of the S outputs. To automate the verification of this conformance relation, it is encoded as the following CSP refinement check:

Definition 2 (cspio verification). $S \sqsubseteq_T (S \triangle ANY(\Sigma_{Io}, Stop)) \,|[\, \Sigma_{IUT} \,]|\, IUT$

The intuition of the right-hand side of the refinement check (Definition 2) is to offer all possible specification traces for the implementation to synchronize and verify the outputs. The direct comparison between S and IUT would not work, since IUT can have, by definition, traces not present in S, because it can accept extra inputs or offer additional outputs after a trace that does not belong to S [6]. Then, $(S \triangle ANY(\Sigma_{Io}, Stop)) \,|[\, \Sigma_{IUT} \,]|\, IUT$ masks IUT traces that should not be verified by *cspio*. It blocks inputs not accepted by S, and the interruption of S on $ANY(\Sigma_{Io}, Stop)$ allows synchronization of output events from IUT that S does not produce, terminating the process. If this interruption happens, then the refinement would be false, as expected. If the resulting process does not produce different outputs from the same inputs, then the refinement is valid and IUT *cspio* S.

The Abstract Test Generator (ATG) provides guided test generation based on the *cspio* conformance relation [14] and CSP traces semantics. Its core idea is to exercise a specification, obtain relevant scenarios as counterexamples of refinement verification, and generate sound test cases from them. From a specification S, traces satisfying a given property can be extracted, with optional selection criteria expressed as test purpose (partial specifications), also defined as CSP processes, that describe the desired aspects for generated tests. A common mechanism involves adding marker events ($MARK = accept.n$) to S to obtain S', then using refinement checking on $S \sqsubseteq_T S'$; counterexamples of the form $ts \frown \langle m \rangle$ yield the scenarios.

A test case (Test Case (TC)) is a CSP process that interacts with the implementation (IUT) through parallel composition, with verdict events *pass*, *fail*, or

inco indicating the outcome. Test case alphabets are dual to those of the *IUT*: test outputs are *IUT* inputs, and *IUT* outputs are test inputs. An important property that should be pursued when generating test cases is that these TCs should not generate false fails. This property, known as soundness, guarantees that when a test execution reaches a fail verdict then the implementation, for sure, does not conform to the specification. The Definition 3 formally defines a sound test case in CSP:

Definition 3 (Soundness).
$\langle fail \rangle \in traces(EXEC \setminus (\Sigma_{I_{IUT}} \cup \Sigma_{O_{IUT}})) \Rightarrow \neg(IUT \ \ cspio \ \ S)$

To build a sound test case from a test scenario, we have to make sure that it records all output events that the specification communicates at each step of the given scenario. The process of building the sound test case is iterative and begins with a default annotated trace *atrace* with the format $\langle (ev_i, out_i) \frown (accept.n, \{\}) \rangle \bullet 1 <= i <= \#ts$ where ev_i is the i^{th} element of *ts* and out_i the corresponding set of output events after performing the trace until ev_{i-1}.

Consistency Analysis. The domain is modeled with CSP events and processes. However, the actual processes that carry out the consistency analysis are discussed next. The core idea is to combine the original requirements in parallel with the domain to build a more detailed specification, which is then used to generate test cases that are consistent in that there are no missing steps necessary for their executions by a test driver.

For a test case to be consistent, all rules expressed in the associated domain model must be satisfied. As presented in Sect. 3.2, these rules can encompass dependencies, compositions, and other relations.

To keep track of which actions are active at each step, we define the process *EXECUTION_HISTORY*, which allows other processes to synchronize with it to check whether an input is currently active. For instance, when evaluating the dependencies of *i_submitmessage*, it could detect that *i_activatedata* was already executed and, therefore, skip its execution. The *EXECUTION_HISTORY* process applies the replicated interleave operator to parametrize a process *LOOP* with all events that may occur from the domain model. The *LOOP* process maintains the record of which actions were executed, synchronizing with the markers *execute* and *executed* whenever an input event is performed on the *REQ* process or by the *DOMAIN* itself.

Listing 1.1 presents the augmented specification that is used to generate concrete test cases: the resulting process of putting the requirements, domain model, and execution history together. The first process, *CONSISTENCY_ANALYZER*, is the result of combining the *DOMAIN* from Sect. 3.2 with the *EXECUTION_HISTORY* process discussed above. With this process, it is possible to reason about consistency since we combine the effects of the rules from the domain model with the expected execution progress. *REQ_MARKED*, in turn, is another intermediate process that adds information about consistency choices directly on the original requirements. Finally,

Listing 1.1. Consistent specification

```
CONSISTENCY_ANALYZER = DOMAIN [|aux_domain_events|]
    ↪ EXECUTION_HISTORY
REQ_MARKED = REQ [+ inter(req_events, choice_events) +]
    ↪ CHOICES
SPEC = (REQ_MARKED [|union(inter(req_events, domain_events),
    ↪ consistency_success_events)|] CONSISTENCY_ANALYZER) \
    ↪ all_aux_events
assert REQ* [T= SPEC*
assert SPEC* [T= REQ*
```

SPEC is the process used later as the specification for test case generation. Instead of capturing only the original and abstract requirements, it now holds more detailed events to generate concrete and consistent test cases.

Since we add behavior to the initial requirements, we must ensure that the original behavior is still preserved. It is important to have a guarantee that any information about the domain does not interfere with the core behavior. While it could be valuable for other scenarios, it is important in our industrial context to ensure that any information added by test analysts (or any other stakeholders) does not tamper with, for instance, third party requirements or critical functionalities. The assertions guarantee not only that the new specification is a refinement of the original requirement, but also that they are in fact equivalent. The refinement, in both directions, is checked after hiding the extra and auxiliary events from both processes.

A test scenario is built from the counterexample produced by checking the model against a specific test purpose. The scenario is then annotated to include all possible outputs for each step. This annotation is important to mark inconclusive results, i.e., outputs that are not relevant for the current scenario, but are possible outcomes defined in the specification [18]. The annotated trace is then supplied to *TC_BUILDER* to build a sound test case. Since the process *TC_BUILDER* is sound [18], the resulting test case is always sound and can then be linearized back to natural language. Each input event becomes a test step, while the corresponding output event is the expected result. We can trace back the action using the *id* from the trace. With its corresponding syntactic structure, we can linearize the action back to English, as seen in Sect. 2.

The total number of scenarios depends on the number of circumstances in the original requirement. It can be calculated by the combination $\binom{n}{k}$ with n as the number of circumstances and $k = 2$, since a circumstance has only two states: it either happens or it does not. There are some cases in which the additional requirements are too obvious or trivial, or even do not make sense. In these cases, the analysts can remove the test cases from the selection after generation.

4.2 Test Case Automation

The output of the proposed test generation strategy is sound and consistent test cases that comply with our CNL. Because these test cases are already consistent, there is no need to insert supplementary setup or reorganize the script since all dependencies and details defined in the domain model are already resolved. Only the mapping to execution methods (which the test adapter recognizes) is lacking. For this matter, we present here a straightforward strategy: direct code mapping.

This straightforward strategy consists of mapping CNL-compliant sentences into their corresponding scripts. This direct association is possible due to the action representation parsed from the sentence. Following the frame theory, we can assign a unique identifier to the fixed frame slots (such as operation and patient) and then map each singular frame to the corresponding script method. We implemented this mapping mechanism using a proprietary framework adopted within the industrial context of our research. Because it is not publicly available, we instead present an analogous implementation using the Java programming language and the UIAutomator framework [1] for Android UI Testing. Listing 1.2 shows an example of this sentence-to-script mapping.

Listing 1.2. Sample Java script mapping

```
@Smartest("press a button")
public void pressButton(String identifier, String description, String
    ↪ text) {
...
  mDevice.findObject(selector).click();
}
```

In Listing 1.2, Line 2, we have a method declaration that is mapped to a sentence by the annotation in Line 1. The parameters have the same name as the slots the associated frame can have. Line 4 shows a method call on the variable *mDevice* that holds the API access to functions related to user interactions with the connected device. The API for the interaction with a device may differ from one automation framework to another.

Because the sentences are CNL-compliant, it does not matter how they were written since they will always be mapped to their corresponding method and the arguments. This mapping must be made for all atomic actions. Since all other actions are compositions of these basic actions, the automation effort is kept at a minimum. The choice of which actions should be atomic is entirely dictated by the team. In our scenario, there is a 1:1 mapping from each API native method available to a single action described in text. Then, assuming that the atomic actions are mapped considering the domain model described above, we can generate a code script that, when executed, gives a pass/fail result.

5 Related Work

Although the related work presented here share some features with ours, particularly considering that the inputs are textual documents in natural language, they do have some fundamental differences that are summarized next.

DASE [23], NAT2TEST [5], TaRGeT [7], UMTG [21], and RTCM [25] provide no reuse of specification/test artifacts. Cucumber [24] allows a simple form of reuse when the step is shared among test scenarios. Our work, instead, addresses reuse to a much larger extent. For instance, in our approach, a test case can be a step of a more elaborate test case, whereas a scenario is not qualified as a possible step for reuse in Cucumber.

A recent work [12] leverages pre-trained Natural Language Processing (NLP) models to generate test scripts by matching natural language descriptions with test methods that locate Graphical User Interface (GUI) elements. Because a pre-trained NLP model is applied, there is a considerable error rate while matching, even though only simple and atomic descriptions are used. In our context, using a CNL for requirements and domain model allows us to create accurate matches, define abstract actions on-the-fly, and ensure soundness and consistency.

The aforementioned strategies that generate test scripts adopt specifications that either match the implementation abstraction level or annotate the abstract model with low-level details. Because our strategy proposes a compositional domain model, details can be further added according to the organization's roles and processes. Besides, because the semantics are formally defined to the atomic level, generated test scripts are verified to be sound and consistent. Most of the above-mentioned approaches focus on generation instead of automation of existing test cases. These approaches rely on formal and well-documented requirements from which they can generate TCs. Unfortunately, up-to-date requirements are seldom available in the non-critical software industry. In addition, to allow a fully mechanized generation of automated test cases, the requirements need to be specified at a lower level of abstraction. None of these approaches provide a (bottom-up) alternative to building a domain model from manually generated TCs and allow consistency checks as we propose here.

None of the cited approaches address test step sequence consistency and dependency notions, with an associated verification mechanism; this is a distinguishing contribution of our approach.

Despite the large scope of our work, we do not support some elements featured in other tools or strategies. For instance, even though we support passing data through parameters, we do not generate test input data. We also do not model timed-based behaviors. Finally, the generation mechanism is tied to the cspio relation and cannot be instantiated in other formalisms as easily as in NAT2TEST, which has an intermediate and hidden formalism.

6 Conclusions

In this work, we present a strategy to generate and automate test cases from requirements written in natural language. Our main contribution is to allow testing teams, that are also involved in a similar industrial perspective, to generate sound and consistent test cases automatically while still writing specifications in natural language (whose meaning is automatically obtained using an underlying, and hidden, formal semantics in CSP). Additionally, with the help of a

dynamically evolving domain model, test teams are not compelled to specify the entire functionality in a single take, allowing them to postpone more concrete descriptions for other roles down the line or when the feature is mature enough to be tested.

In summary, the scope of this work encompasses requirement and domain model specifications, and the generation of test scripts, including the necessary artifacts in between. Specification, verification, and generation are supported. Because the semantics are formally defined and mechanically verified by custom tools, we ensure two important properties automatically: soundness and consistency. While the former property is well-known and regarded in the literature for test generation, the latter is a major contribution of our work, which ensures that the generated test cases can be executed on a concrete implementation. This consistency notion is derived from the information present in the domain model (dependencies, cancellations, details, etc.) while the generation mechanism ensures that these relations do not change the behavior specified by the requirements. Another major contribution is the definition of a CNL flexible enough for describing both requirements and the corresponding test cases.

6.1 Ongoing and Future Work

Regarding ongoing and future work, it is not uncommon to find teams that do not follow the traditional specification phase and skip it until they need to write test cases. Because of this, there are no requirements to rely on, which blocks test design teams from using established strategies that generate test cases while ensuring soundness. In this situation, the only relevant artifacts are the existing test cases or scripts. Therefore, a possible alternative is to apply a reverse engineering process: the existing scripts can be used to abstract test case descriptions in a CNL, from which use cases can also be derived, and ultimately, into high-level requirements.

While the use of a CNL allows us to precisely define parsing rules and formal semantics, authoring requirements that fully comply with CNL syntactic and semantic constraints remains a non-trivial task. Even though the proposed editor assists users by highlighting errors and providing autocomplete suggestions, some users still find the process cumbersome or restrictive compared to their accustomed modes of expression. This usability challenge represents a key barrier to broader industrial adoption.

In this context, we propose investigating the use of Large Language Models (LLMs) [20,22] to interpret freestyle natural language descriptions extracted from unstructured requirement documents and transform them into structured representations in the form of *frames* and *slots*. Instead of relying solely on a constrained input format, LLMs could automatically derive the structured information that underpins formal reasoning. While such automatic mappings cannot guarantee complete unambiguity, they hold the potential to significantly enhance productivity and reduce the cognitive overhead associated with CNL authoring. Moreover, this capability would enable the parsing of legacy requirement descriptions.

Then, future research could explore two integration strategies. First, a *direct mapping approach* could aim to use LLMs to infer the frames and slots directly from uncontrolled natural language. Second, an *indirect approach* could leverage LLMs as CNL translators, converting informal requirement sentences into grammatically and semantically valid CNL constructs. This would preserve the interpretability and verifiability of the CNL framework while automating its most labor-intensive aspects.

Acknowledgments. We thank Motorola Mobility, a Lenovo company, for the long-term partnership and for the financial support that has allowed the applicability of research results on test case generation and automation in an industrial context.

References

1. Android: Android uiautomator description (2015). http://developer.android.com/tools/help/uiautomator/index.html. Accessed 25 Mar 2015
2. Angelov, K.: The Mechanics of the Grammatical Framework. Chalmers Tekniska Hogskola (Sweden) (2011)
3. Arruda, F., Barros, F., Sampaio, A.: Extended version: deriving sound test scripts from requirements written in a controlled natural language. https://github.com/fmca/sbmf2025 (2025)
4. Brinksma, E.: A theory for the derivation of tests. In: Proceedings of the 8th International Conference Protocol Specification, Testing and Verification, pp. 63–74. North-Holland (1988)
5. Carvalho, G., et al.: NAT2TESTSCR: test case generation from natural language requirements based on SCR specifications. Sci. Comput. Program. **95**, 275–297 (2014)
6. Cavalcanti, A., Hierons, R.M., Nogueira, S., Sampaio, A.: A suspension-trace semantics for CSP. In: 10th International Symposium on Theoretical Aspects of Software Engineering, TASE 2016, Shanghai, China, July 17–19, 2016, pp. 3–13 (2016). https://doi.org/10.1109/TASE.2016.9
7. Ferreira, F., Neves, L., Silva, M., Borba, P.: Target: a model based product line testing tool. Tools Session of CBSoft (2010)
8. Fillmore, C.J.: The case for case'in bach & harms (eds.) universals in linguistic theory. Holt, Rinehart, and Winston (1968)
9. Gaudel, M.-C.: Testing can be formal, too. In: Mosses, P.D., Nielsen, M., Schwartzbach, M.I. (eds.) CAAP 1995. LNCS, vol. 915, pp. 82–96. Springer, Heidelberg (1995). https://doi.org/10.1007/3-540-59293-8_188
10. Grechanik, M., Xie, Q., Fu, C.: Maintaining and evolving GUI-directed test scripts. In: Proceedings of the 31st International Conference on Software Engineering, ICSE 2009, pp. 408–418. IEEE Computer Society, Washington, DC, USA (2009). https://doi.org/10.1109/ICSE.2009.5070540
11. Gruzitis, N., Paikens, P., Barzdins, G.: Framenet resource grammar library for GF. In: International Workshop on Controlled Natural Language, pp. 121–137. Springer (2012)
12. Li, C.: Mobile GUI test script generation from natural language descriptions using pre-trained model. In: 2022 IEEE/ACM 9th International Conference on Mobile Software Engineering and Systems (MobileSoft), pp. 112–113. IEEE (2022)

13. Minsky, M.: A framework for representing knowledge. The Psychology of Computer Vision (1975)
14. Nogueira, S., Sampaio, A., Mota, A.: Test generation from state based use case models. Formal Asp. Comput. **26**(3), 441–490 (2014)
15. Rafi, D.M., Moses, K.R.K., Petersen, K., Mäntylä, M.V.: Benefits and limitations of automated software testing: Systematic literature review and practitioner survey. In: 2012 7th International Workshop on Automation of Software Test (AST), pp. 36–42. IEEE (2012)
16. Ranta, A.: Grammatical framework: Programming with multilingual grammars, vol. 173. CSLI Publications, Center for the Study of Language and Information Stanford (2011)
17. Roscoe, B.: An operational semantics for CSP (1986)
18. Sampaio, A., Nogueira, S., Mota, A., Isobe, Y.: Sound and mechanised compositional verification of input-output conformance. Softw. Test., Verif. Reliab. **24**(4), 289–319 (2014). https://doi.org/10.1002/stvr.1498
19. Tretmans, J.: Test generation with inputs, outputs and repetitive quiescence. Softw. Concepts Tools **17**(3), 103–120 (1996)
20. Vaswani, A., et al.: Attention is all you need. In: Advances in Neural Information Processing Systems, vol. 30 (2017)
21. Wang, C., Pastore, F., Goknil, A., Briand, L.C., Iqbal, Z.: UMTG: A toolset to automatically generate system test cases from use case specifications. In: Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, pp. 942–945. ESEC/FSE 2015, ACM, New York, NY, USA (2015).https://doi.org/10.1145/2786805.2803187
22. Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q.: Software testing with large language models: survey, landscape, and vision. IEEE Trans. Softw. Eng. **50**(4), 911–936 (2024)
23. Wong, E., Zhang, L., Wang, S., Liu, T., Tan, L.: Dase: document-assisted symbolic execution for improving automated software testing. In: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineeringm vol. 1, pp. 620–631. IEEE (2015)
24. Wynne, M., Hellesoy, A.: The cucumber book: behaviour-driven development for testers and developers. Pragmatic Bookshelf (2012)
25. Yue, T., Ali, S., Zhang, M.: RTCM: a natural language based, automated, and practical test case generation framework. In: Proceedings of the 2015 International Symposium on Software Testing and Analysis, pp. 397–408. ISSTA 2015, ACM, New York, NY, USA (2015). https://doi.org/10.1145/2771783.2771799